Latent Space Map for Visual Utilization of Generated Data

Yang Zhang, Jie Li, and Wei Zeng

***Abstract*—Samples produced by generative models, called Generated Samples (GSs), have become a critical supplement to those collected from the real world in data-centric applications. Domain experts typically randomly collect many GSs and manually select a few of interest for applications. However, the methodology lacks guidance to locate desirable ones that exhibit specific features or adhere to application-oriented metrics among infinite generable candidates. These samples are generally concentrated in a few small regions of the generative model's latent space, called Generative Latent Space (GLS). This paper presents *Latent Space Map* that projects a GLS onto a plane to help users locate regions rich in desirable GSs. Our research revolves around two challenges in constructing the map. First, many GSs in a GLS are low-quality and useless for applications. Excluding them from the projection is challenging for their irregular distribution. We employ a Monte Carlo-based method to capture a manifold for projection, where high-quality GSs are mainly distributed. Second, the GLS is high-dimensional and unbounded, complicating the projection. We design a manifold projection method that endows the map with desirable characteristics to achieve high display accuracy and effective pattern perception for users freely observing the manifold. We further develop a system integrating Latent Space Map to aid in GS selection and refinement. Real-world cases, quantitative experiments, and feedback from domain experts confirm the usability and effectiveness of our approach.**

***Index Terms*—Latent space visualization, manifold sampling, generative model, generated data utilization, VAE, GAN.**

I. Introduction

GENERATIVE models, such as Variational Autoencoders (VAEs) [1] and Generative Adversarial Networks (GANs) [2], are foundational deep learning techniques. These models have the ability to generate an infinite number of samples, known as generated samples (GSs), which closely resemble training data but exhibit richer features. This characteristic makes GSs valuable complements to real-world data in many scenarios, leading to reduced data acquisition expenses and increased efficiency in data-centric applications. For example, designers often use GSs for designing new industrial products [3], while machine learning experts employ GSs for data augmentation to facilitate model development [4].

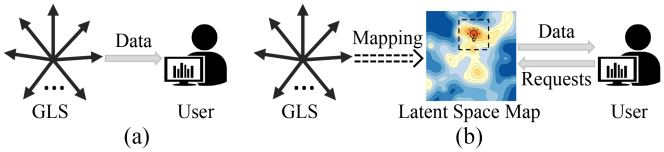

Fig. 1. Comparison of our approach with existing workflow. (a) Users randomly gather GSs from the GLS in the existing workflow. (b) Our approach involves constructing a Latent Space Map for the GLS, which offers users visual guidance to identify regions rich in desirable GSs.

GSs are essentially samples generated from the generative latent space (GLS) of a model, while GLS is a high-dimensional embedding space that encompasses all possible GSs. Existing works that utilize GSs typically employ random sampling to collect a predetermined number of GSs from a GLS [5], [6], as illustrated in Fig. 1(a). However, users often seek GSs that exhibit desirable features or adhere to application-oriented metrics, which are typically concentrated in specific regions of a GLS. There is a need for visual guidance on adjusting the sampling settings to locate these regions and obtain such desirable GSs.

To meet the need, we introduce an interactive visualization approach for utilizing GSs. At the core of the work, we propose *Latent Space Map*, which maps GLS onto a two-dimensional plane, with each position corresponding to a GS and color-coded according to application-specific metrics. With the map, users can visually identify regions abundant in desirable GSs by analyzing the color distribution of GSs, as illustrated in Fig. 1(b). The construction of Latent Space Map revolves around two key challenges. First, a substantial portion of GSs within a GLS are of low quality and useless for applications. High-quality GSs are characterized by (i) fidelity, reflecting their realism, and (ii) diversity and mode coverage, capturing the variability of the data distribution [7]. Excluding low-quality GSs from the map enhances data validity but is challenging due to their irregular distribution [8]. Second, the high-dimensional and unbounded nature of a GLS complicates mapping onto a plane. In particular, the map should accurately reflect the characteristics of a GLS and enable users to explore the manifold at various scales and regions.

We design a two-stage approach for constructing the Latent Space Map. First, we leverage pre-collected high-quality GSs as references and employ a Monte Carlo-based sampling

Received 17 January 2025; revised 11 September 2025; accepted 17 September 2025. Date of publication 25 September 2025; date of current version 10 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372321, and in part by Guangxi Science and Technology Program under Grant AB25069470. Recommended for acceptance by I. Viola. *(Corresponding author: Jie Li.)*

Yang Zhang and Jie Li are with the College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: jie.li@tju.edu.cn).

Wei Zeng is with the Hong Kong University of Science and Technology, Guangzhou 511453, China, and also with the Hong Kong University of Science and Technology, Hong Kong 999077.

This article has supplementary downloadable material available at https://doi.org/10.1109/TVCG.2025.3614247, provided by the authors.

Digital Object Identifier 10.1109/TVCG.2025.3614247

1077-2626 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

method [9] to compute a Riemannian metric that captures the geometric structure of a manifold where high-quality GSs are commonly found [10], [11] (Section V). By simulating the movement of these references within GLS, we can identify high-quality GSs on the manifold and outline its boundary. This approach ensures the map's reliability with high-quality GSs, meanwhile accelerating the computation process. Second, we introduce an improved visualization technique for manifold projection with various visual enhancements (Section VI). Specifically, we employ Supervised AutoEncoder (SAE) [12] to generate 2D embeddings of reference samples for projection and treat the task of finding the associated GS for a specific position on the map as an optimization problem. This method overcomes the limitations of existing visualizations that rely on a fixed number of precomputed GSs (e.g., [6], [13]) or a 2-dimensional GLS (e.g., [14], [15]). We incorporate interactions like zooming and panning, along with high display accuracy to help users explore the manifold and identify desired GSs without manual adjustment of sampling settings.

On the basis of Latent Space Map, we develop a visualization system to bridge the gap between the map and real-world applications (Section VII). The system comprises a Global View for exploring the Latent Space Map and selecting GSs, along with a Local View that highlights GSs differing from user-selected target GS in specific features. Both views offer operations to simplify GS selection, facilitate GS comparison, and improve the identification of desirable GSs. The map and the visualization system have been evaluated through both qualitative and quantitative assessments (Section VIII). We present case studies involving real-world datasets from scenarios related to airfoil design and model development. We also conduct quantitative experiments to assess the effectiveness of our method for capturing the manifold, and the results indicate that our approach surpasses existing methods in terms of the quality of collected GSs. Feedback gathered from domain experts through interviews confirms the system's usability and effectiveness in practical scenarios.

In summary, this work contributes a visualization approach to facilitate the utilization of GSs.

- We introduce the Latent Space Map, a novel visualization method that projects the manifold of high-quality GSs within a GLS onto a plane with desirable characteristics.
- We develop a visualization system that integrates Latent Space Map with various interactive features, providing a unified GS utilization process for practical applications.
- We conduct case studies to illustrate how the visualization system aids in identifying desirable airfoil designs and diagnosing and improving model classifiers.

II. Background

This section presents a brief introduction of critical concepts, including Generated Sample (GS) and Generative Latent Space (GLS), to facilitate our discussion in this paper.

Generative models map the GLS to the data space, ensuring that generated samples conform to the real data distribution. Each latent vector $z \in \mathbb{R}^n$ in GLS can be converted into a GS $x \in \mathbb{R}^m$ through the model. In VAEs, the encoder $q(z \mid x)$ maps input x to a latent vector z, while the decoder $p(x \mid z)$ reconstructs x from z. Training maximizes the evidence lower bound (ELBO) [1], which encourages the encoder to approximate the posterior distribution and the decoder to reconstruct the input from the latent vector. In GANs, the generator $G(z)$ takes noise z as input to generate samples $\hat{x}$, while the discriminator $D(x)$ differentiates between synthetic samples $\hat{x}$ and real samples x. Training minimizes the adversarial loss, which encourages the generator to maximize $D(G(z))$ to produce realistic samples and the discriminator to maximize $1 - D(G(z))$ to classify the produced samples as fake.

Generative models typically assume that GLS is a smooth manifold and are trained to map training data onto this structure [16]. Here, a manifold is a geometric structure that locally resembles Euclidean space. Nevertheless, not all GSs collected from a GLS are guaranteed to be of high quality and exhibit useful features for practical applications. Studies have identified the presence of "dead zones" [8] that contain low-quality GSs, often characterized by excessive noise or improbable features in the real world (see Appendix I). Therefore, effectively identifying high-quality GSs is essential for enhancing their utility, presenting a primary challenge for this work. Fortunately, previous work [17] has revealed that high-quality GSs tend to be distributed along a low-dimensional Riemannian manifold. Moreover, by defining local coordinates, one can capture the geometric properties that reflect the intrinsic data structure of the GSs. This allows for the utilization of geometric metrics to improve the sampling process of gathering GSs from a GLS.

A second challenge arises in visualizing GLS while capturing its key characteristics. First, the number of GSs in a GLS is infinite due to its unbounded scope, referred to as **Unboundedness**. Second, each position in a GLS corresponds to a latent vector that can be transformed into a GS, referred to as **Generativity**. Third, adjacent GSs exhibit similarity in features, with this similarity gradually decreasing as the distance between them increases [13], referred to as **Similarity**. These characteristics contribute to making a GLS resemble a learned database with an infinite supply of GSs for applications. Therefore, ensuring that the Latent Space Map aligns with these characteristics presents a nontrivial challenge.

III. Related Works

A. Sampling in Generative Latent Space

Finding high-quality samples with broad feature diversity in a GLS is essential for GS utilization. Existing techniques for sampling in GLS can be categorized into three groups.

Prior-based methods: Many generative models (e.g., [1], [2], [18]) assume that their inputs follow a Gaussian prior. As such, one can randomly collect latent vectors based on this prior distribution, and input them into the model to generate samples. However, Tomczak and Welling [19] pointed out that simple priors often result in the generation of low-quality samples. Several methods have been proposed to improve the quality of GSs. One approach is to use a Gaussian mixture model instead of a simple Gaussian prior to better capture the complexity of real

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

data distribution [19], [20]. Another strategy involves learning the prior directly from the training data [21], [22]. However, mismatches between real data distribution and the prior can lead to the presence of dead zones that contain low-quality samples [8], which can significantly impact the sampling outcomes.

Interpolation-based methods: Given a set of GSs, new samples can be generated using linear combinations of their latent vectors as inputs [5], [8]. However, linear interpolation may traverse dead zones, resulting in the generation of low-quality samples. White [8] introduced spherical linear interpolation for collecting GSs along a circular path on a hypersphere. The method decreases the probability of encountering dead zones but does not completely eliminate this possibility. Arvanitidis et al. [23] regarded GLS as a geometric manifold and collected GSs along the geodesic line. Although this approach shows improved performance, it suffers from high time complexity. Alternatively, other studies have identified semantic directions for feature variations and collected GSs through linear interpolation [24], [25]. Their objective is to find GSs with specific features, which differs from our goal of collecting high-quality GSs. Instead, this work focuses on capturing the manifold where high-quality GSs are distributed and collecting samples from regions within that manifold.

Monte Carlo-based methods: Monte Carlo techniques can be used for estimating distributions, and they have been employed for sampling in GLS [9], [26], [27]. These studies estimate the posterior of the learned latent vectors using the provided training set and gather GSs based on the estimator. By avoiding direct sampling from the prior that may not align with the posterior, these methods enhance the quality of GSs. For example, Girolami and Calderhead [9] inferred the posterior by simulating samples that move in a physical system, guided by geometric metrics towards regions of high probability in the posterior distribution. However, the high computational overhead hinders their applicability in our system, which requires real-time collection of numerous GSs. To improve sample efficiency, an emerging trend involves integrating posterior estimation into model training by incorporating a series of mappings [28], [29], [30]. This allows the model to convert randomly collected latent vectors to GSs that adhere to the posterior distribution through these mappings. Despite their efficiency, these methods are tailored to specific models (such as VAEs), which contrasts with our objective of a method suitable for popular generative models.

We draw inspiration from Monte Carlo methods to develop a technique for collecting GSs from GLS. Specifically, our approach conceptualizes sampling as the motion of particles [9], which is influenced by the geometric metric of GLS. We introduce a sampling strategy to overcome the computational speed limitations associated with the geometric metric. As a result, our approach is computationally efficient, model-agnostic, and not reliant on a prior.

B. Latent Space Visualization

Latent space visualization is employed to facilitate the use of GSs. A common approach involves projecting pre-collected GSs through dimensionality reduction (DR) [5], [6], [31], [32]. These DR algorithms typically arrange GSs with similar attributes close to each other in the projection and provide visual cues for selecting GSs. An alternative approach is to map GSs onto predefined grids, ensuring that GSs in neighboring grids share similar characteristics [33], [34]. This technique prevents GSs from overlapping and maximizes the use of space in the projection. However, these methods only utilize pre-collected GSs without incorporating feedback to dynamically adjust the sampling range.

Other studies have focused on visualizing a GLS rather than pre-collected GSs to harness its infinite potential GSs. One straightforward approach is to train generative models with 2-dimensional GLSs, which can be feasibly visualized on a 2D plane. This allows GSs to be directly represented on a plane, with each position colored based on the user-specified metric value of GSs. For example, GANLab [15] colors sample positions based on the discriminator's output to show the GAN training process. The approach has been utilized for layout selection [14] and skeletal gesture optimization [35]. However, it has been demonstrated that when generating GSs with complex features like images, a high-dimensional GLS (e.g., $\geq$64) is necessary to ensure high-quality GSs [36]. The constraint of depending on 2-dimensional GLS impacts the usability of these tools.

Visualizing high-dimensional GLS can be accomplished by mapping 2D coordinates to a high-dimensional latent vector. iLAMP [37] effectively performs back-projection for LAMP [38] by considering local neighborhoods. However, iLAMP is tailored specifically for LAMP and may not be suitable for other projection techniques. Amorim et al. [39] approached back-projection as an interpolation challenge of global mapping from low to high dimensions. This interpolation method relies on Euclidean distance, which may result in low-quality GSs in dead zones [8]. Besides back-projection, GLS decoupling presents another strategy by creating a GLS where each dimension corresponds to a semantic direction, enabling gradual changes in features among GSs. Recent studies [40], [41] chose two directions as the axes of a GLS, ensuring that each position on the plane corresponds to a GS with adjacent GSs exhibiting similar features. Liu et al. [13] enabled users to select semantic directions for visualizing the GSs in a GLS. However, this approach may not capture feature changes driven by other directions on a plane and does not guarantee that all GSs are of high quality.

In this study, we project a high-dimensional GLS onto a 2D plane, where each position represents a high-quality GS. This plane is equipped with diverse operations to enable users to observe GSs across various regions of a GLS. Our system offers improved usability through 1) constructing a plane that retains the key characteristics of a GLS, and 2) ensuring that all GSs on the plane are of high quality.

IV. Requirement Analysis and Approach Overview

This section introduces application scenarios (Section IV-A), followed by the requirement analysis for effective GS utilization (Section IV-B). Lastly, we provide an overview of our approach (Section IV-C).

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

A. Application Scenarios

GSs serve as a valuable supplement to real-world data samples, in areas such as model development and product design. Here, we outline how domain experts leverage GSs to distill the design requirements of our approach.

Model Development: The use of GSs can augment training sets and improve training outcomes [42], [43], [44]. There are typically two approaches for effectively utilizing GSs. First, model developers can randomly select GSs from a GLS and incorporate them directly into the training set [4], [45]. Second, if the model performs poorly on specific sample subsets, such as samples from a particular class or with distinct features, model developers can gather GSs from a subset that is closely related in a small region of a GLS [46], [47].

Product Design: GSs can be used to automatically generate solutions that enhance efficiency [48]. There are also two strategies for leveraging GSs in this context. First, designers can randomly generate design candidates and then filter them using quantitative or subjective criteria [3], [49]. Second, designers can gather solutions that deviate from a preliminary design target in a few specific features [33], [50]. They can then select a few solutions with optimal feature variations to refine the target design.

B. Requirement Analysis

This study was conducted in collaboration with two independent experts specialized in product design (E1) and machine learning (E2). E1 (male, aged 27) is a PhD student with expertise in aerodynamics and airfoil design. E2 (male, aged 40) is a postdoctoral researcher in machine learning and has experience with generative model-based data augmentation. Both experts are affiliated with the university of the first author but are not directly involved in our research. To better understand their needs, we conducted semi-structured interviews with the experts, each lasting between 30-60 minutes.

From the interviews, we have identified three design requirements for effective GS utilization:

R1. Filtering out low-quality GSs: Many GSs in a GLS contain noise or exhibit features that are unrealistic in real-world scenarios [8]. Including them in the analysis loop or integrating them into the training dataset is futile and could result in biased analytical outcomes and reduced model performance. Therefore, excluding low-quality GSs can effectively prevent interference in the utilization process.

R2. Mapping a high-dimensional GLS to a 2D plane: Although the GLS encompasses all possible GSs, its high dimensionality makes it difficult to understand. To enhance the usability of a high-dimensional GLS for diverse tasks, experts need to effectively interpret and extract insights from GSs contained within the GLS. This requires mapping the high-dimensional GLS to a 2D plane while meeting several complex requirements, including: **R2.1**) ensuring alignment with the characteristics of a GLS (i.e., unboundedness, generativity, and similarity), **R2.2**) offering accurate and efficient map operations, such as pan and zoom, and **R2.3**) optimizing visual effects to enhance display precision and perception.

R3. Developing an effective visualization system: The effective utilization of GSs relies on an interactive visualization system that seamlessly integrates the processes of collection, selection, and refinement, as depicted in Section IV-A. The system should offer users a holistic global view of a GLS for selection, while also providing tailored local views of individual GSs for refinement. Furthermore, the visualization system should support user interactions, allowing users to delve into specific GSs and facilitate easy comparisons between them.

C. Approach Overview

An overview of our approach is presented in Fig. 2, which encompasses three components to address the requirements:

1) *Manifold Sampling (Section V):* We design a Monte Carlo -based sampling method to capture the manifold where high-quality GSs are distributed, as shown in Fig. 2(a). Particularly, we leverage a group of pre-collected high-quality GSs as references and simulate their movement in a GLS with constraints. Each position that a reference can reach corresponds to a collectible high-quality GS on the manifold, outlining the manifold's boundary. Thus, one can collect GSs from this manifold rather than the whole GLS, ensuring that the GSs collected are high-quality and improving validity for their utilization (**R1**).
2) *Latent Space Map (Section VI):* We design a new projection method, namely Latent Space Map, to enable users to observe all GSs within a GLS, as shown in Fig. 2(b). Specifically, we project the manifold onto a 2D plane in which each position corresponds to a high-quality GS and color each position based on its metric tailored to applications. In this way, this plane successfully maintains the characteristics of unboundedness and generativity, while also improving the validity of the visualization (**R2.1**). Upon user operation, new GSs are collected in the current view and the plane will be redrawn to ensure high accuracy (**R2.2**). We also constrain the GSs arranged on the plane continuously with their features or certain labels to enhance pattern perception (**R2.3**).
3) *Visualization System (Section VII):* We design a visualization system to bridge the gap between Latent Space Map and its practical applications. The system comprises two primary views, as shown in Fig. 2(c). In addition to a global view for examining Latent Space Map and selecting initial GSs, there is also a local view displaying GSs that differ from a user-selected GS. Both views are equipped with various operations for facilitating GS selection and comparison and enhancing effectiveness in identifying patterns (**R3**).

V. Manifold Sampling

This section introduces the methodology for constructing a manifold where high-quality GSs are distributed (Section V-A), followed by a sampling method designed to collect GSs on the manifold (Section V-B), to meet requirement **R1**.

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

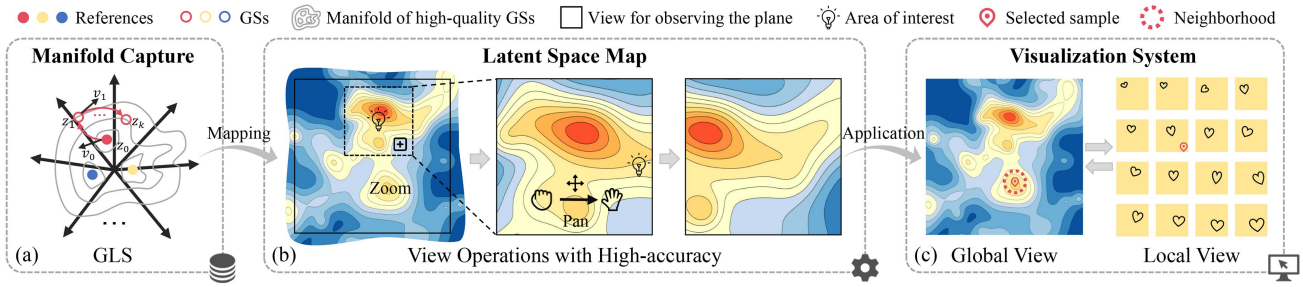

Fig. 2. The proposed approach mainly consists of three components. (a) Capturing a manifold with references for sampling. (b) Constructing Latent Space Map with view operations to allow for free map exploration. (c) Developing a system for the practical application of the Latent Space Map.

TABLE I
COMPARISONS OF FID AND IS OF GSS COLLECTED USING THREE METHODS ON SIX DATASETS

DataSet	Prior-based		MC-based		Reference-based	
	FID↓	IS↑	FID↓	IS↑	FID↓	IS↑
Airfoil (VAE, d = 10)	118.93	-	46.81	-	46.30	-
Fashion (VAE, d = 10)	63.61	2.78±0.10	46.42	2.97±0.06	46.46	2.96+0.09
MNIST (SNGAN, d = 32)	32.04	2.61±0.05	-	-	16.60	2.64±0.04
Cifar10 (BigGAN, d = 208)	6.22	8.47±0.10	-	-	5.94	8.63±0.07
Cifar100 (StyleGAN, d = 128)	20.55	6.88±0.27	-	-	18.46	7.26±0.14
Tiny ImageNet (ECGAN, d = 100)	22.67	15.06±0.34	-	-	18.32	18.44±0.17

TABLE II
COMPARISONS OF THE TIME (IN SECONDS) TAKEN TO COLLECT GSS WITH AND WITHOUT THE PRE-CALCULATION OF JACOBIAN MATRICES FOR DIFFERENT GLS DIMENSIONALITIES

	With Pre-calculation				Without Pre-calculation
	S=50	S=100	S=250	S=500	S=1
d=10	0.1897	0.2473	0.7245	1.2130	63.1968
d=32	0.2174	0.3315	0.8778	1.4871	-
d=64	0.3925	0.6520	1.8207	3.2101	-
d=128	0.6346	1.0739	2.7901	5.2978	-
d=256	1.9944	3.5596	6.1463	14.1681	-
d=512	7.1883	13.6163	34.4405	77.2328	-

A. Manifold Construction

Research has proven that high-quality GSs are mainly on a Riemannian manifold within the GLS [10], [11]. Building on this insight, the objective is to capture the geometric structure of the manifold by computing the Riemannian metric [51]. The metric characterizes the local geometry of the manifold and guides subsequent sampling strategies.

We utilize a set of pre-collected high-quality samples as references to characterize the manifold. Given the references and their latent vector z, Riemannian metric $G(z)$ can be computed using the definition proposed by Louis [52], which describes local geometric structure of z in the GLS, as follows:

$$G^{-1}(z) = \sum_{i=1}^{N} L_i^T L_i \exp\left(-\frac{\|z - z_i\|_2^2}{\gamma^2}\right) + I, \qquad (1)$$

where N is the number of the references, z_i is the latent vector of ith reference, L_i is a lower triangular matrix, $\gamma \in \mathbb{R}$ is a hyperparameter (1.0 by default, see Appendix V), and I is a regularization term. γ scales the impact of references, where smaller values assign less weight to distant references. We can get L_i through the Cholesky decomposition of $G(z_i) = J_{z_i}^T J_{z_i}$ (J_{z_i} is Jacobian matrix, see Appendix II).

For reference selection, samples from the training set are typically suitable, since they represent key features likely to be encountered in real-world scenarios. Notably, there are no restrictions on training samples. Users can choose any set of samples as references, while GSs on the manifold captured by our approach are closely aligned with the reference distribution. In addition, obtaining the latent vectors of these references in the GLS is essential for calculating $G(z)$. Models equipped with an encoder, such as VAEs, can easily obtain latent vectors by generating embeddings of the training set. For models lacking an encoder, like GANs, we employ the inversion technique [53] to map samples back to the GLS and derive their embeddings.

The computation of $G(z)$ requires time-consuming layer-by-layer derivatives [23], [54], inapplicable to our scenario that involves real-time collection of GSs for rendering visualizations. To accelerate the computation process, we implement three optimizations: (1) pre-calculating and storing all Jacobian matrices of references, (2) querying the N-nearest references rather than all references for calculation, and (3) constructing a KDTree that stores the coordinates of all references to accelerate nearest neighbor queries. Experimental results prove that the pre-calculation of Jacobian matrices significantly improves sampling efficiency (see Table II) and setting $N = 100$ is sufficient to achieve favorable performance (see Fig. 11(b)).

B. Reference-Based Sampling

Once the manifold is characterized by $G(z)$, we employ Riemannian Hamiltonian Monte Carlo [9] to sample GSs from the manifold. Specifically, the collected GSs should (1) be co-distributed with the references, making them located on the manifold for high quality, and (2) remain distant from the references, making them different from the references for feature diversity. Therefore, we model the GLS as a physical system in which each reference can move under the influence of potential and kinetic energies to reach a series of positions, denoted as $z_0, z_1, \ldots, z_k$ (the initial position z_0 is the latent vector of the reference), as in Fig. 2(a). The potential energy attracts the reference toward its initial position for (1), while the kinetic energy drives the reference away from its initial position for (2).

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

All reachable positions lie on the manifold, corresponding to high-quality GSs with diverse features.

Let x be a reference and z be its initial position (i.e., latent vector). Our goal is to 1) define its potential energy $U(z)$ and kinetic energy $K(z, v)$ (v is its velocity), 2) make z move based on these two energies, and 3) collect its waypoints for sample generation. On the basis, $U(z)$ and $K(z, v)$ are computed as:

$$U(z) = -\log p(z),$$
$$K(z, v) = \frac{1}{2} v^T G^{-1}(z) v, \tag{2}$$

where $v \sim N(0, G(z))$ follows a normal distribution with zero mean and covariance matrix $G(z)$, and $p(z)$ is probability of z. In VAEs, $p(z)$ corresponds to $p(z \mid x)$, while in GANs, it is related to $D(G(z))$ (symbols are defined in Section II). Higher values of these terms indicate that the GSs are closer to the references, resulting in lower potential energy. Conversely, lower values imply that GSs are farther from the references, increasing the potential energy. We can solve the Hamiltonian equations $H(z, v) = U(z) + K(z, v)$ [9] to obtain two functions $z(t)$ and $v(t)$ that return z and v at a time step t (see Appendix III). All the waypoints $z(0), z(1), \cdots, z(t)$ can be regarded as GSs sampled from the manifold.

Based on the above process, an infinite number of GSs can be sampled from this manifold. We design an adjustable sampling strategy for any display region based on pre-collected references, as illustrated in Algorithm 1. Let M be the set of pre-collected references, S be the set of GSs to be collected, and K be the number of time steps each reference undergoes. We take $z \in M$ as z_0 (line 5), sample v_0 (line 6), and iteratively calculate $z(k)$ and $v(k)$ by resolving Hamiltonian equations (lines 9-10). We further define an acceptance probability $\alpha(k)$ to determine whether $z(k)$ is retained, as follows:

$$\alpha(k) = \min\left(1, \frac{\sqrt{\det G(z(k))}}{\sqrt{\det G(z(k+1))}}\right). \tag{3}$$

The smaller $\det G(z)$, the better the quality of GSs around z [54]. $\alpha(k)$ prevents the reference from moving to a position surrounded by lower-quality GSs. After K time steps ($K = 50$ by default, see Appendix V), we add $z(K)$ to S (line 12). Thus, we can collect high-quality GSs based on specific references or their subset. This approach balances that of preserving the manifold structure with promoting feature diversity.

VI. Latent Space Map

After constructing the manifold where high-quality GSs are distributed, the next step is to map the manifold onto a 2D plane, referred to as Latent Space Map, to enhance the usability and effectiveness of GLS (**R2**). This section first discusses the characteristics required for the mapping (Section VI-A), followed by visual and interaction designs (Sections VI-B-VI-D) that align with these characteristics.

Algorithm 1: Reference-Based Sampling Algorithm.

1: **Input:** M, K
2: **Output:** A set of collected GSs S
3: Set $S \leftarrow$ empty array
4: **for** each $z \in M$ **do**
5: $\quad z(0) \leftarrow z$
6: $\quad$ Sample $v(0) \sim N(0, G(z(0)))$
7: $\quad$ **for** $k \leftarrow 1$ **to** K **do**
8: $\quad\quad v(k) \leftarrow v(k-1) + \Delta v(k-1)$
9: $\quad\quad z(k) \leftarrow z(k-1) + \Delta z(k-1)$
10: $\quad\quad$ Accept $z(k)$ with probability $\alpha(k)$ in Equation 3
11: $\quad$ **end for**
12: $\quad$ Append $z(K)$ to S
13: **end for**
14: **Return** S

A. Mapping Characteristics

The intended Latent Space Map is essentially a projection of a GLS onto a plane. Therefore, it is crucial that Latent Space Map shares common characteristics with the original GLS (see Section II).

- *C1. Unboundedness:* A GLS is high-dimensional and unbounded. Therefore, the Latent Space Map is also unbounded and supports continuous zooming to allow users to observe the GLS at different scales.
- *C2. Generativity:* Each position in the GLS can be converted to a GS by the model. Therefore, the Latent Space Map contains infinite GSs due to its unboundedness and allows users to get a valid GS at any location.
- *C3. Similarity:* The differences in features between GSs increase with their Euclidean distance in the GLS. Therefore, the Latent Space Map should make proximal GSs exhibit more similar features.

Besides, the "map" metaphor requires Latent Space Map to have two additional critical characteristics:

- *C4. Validity:* All GSs in the map should be of high quality, with minimal noise and a distribution similar to those gathered from real-world data. This ensures that the visualization rendered on the plane accurately reflects useful application results.
- *C5. Consistent High Accuracy:* The map should allow for multi-level zooming for fine-grained exploration of local regions. GSs should be progressively collected from specific regions to maintain a high accuracy.

We implement several mechanisms to ensure that Latent Space Map conforms to these characteristics. The following sections describe these mechanisms in detail.

B. Manifold Projection

We opt to project the manifold rather than the whole GLS onto the plane, to ensure that all GSs in the map are high-quality (**C4**). Leveraging pre-collected references to capture the manifold simplifies the mapping process. By projecting the references directly onto a 2D plane, we can observe a low-dimensional representation of the manifold: areas with references are part of

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

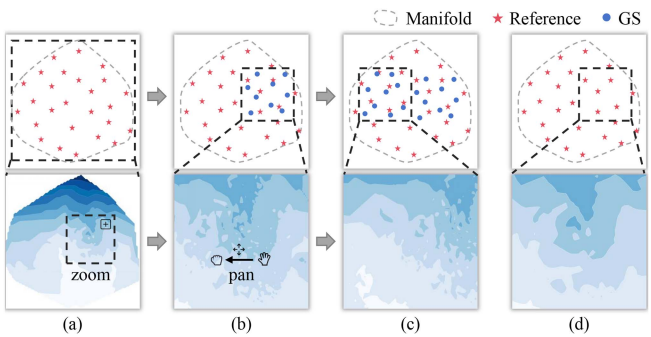

Fig. 3. Illustration of how to construct Latent Space Map. (a) Initialization of the map by projecting references and creating a contour plot based on user-defined metric values. (b-c) Adjusting the view by panning or zooming into a specific area with limited references. New high-quality GSs are obtained from the manifold using the references in that area. These GSs are incrementally projected, and the contour plot is redrawn to maintain accuracy. (d) Displaying the contour plot using a set number of references. The accuracy decreases when zooming into a small area with only a few references.

the manifold, while areas without references can be seen as dead zones, as those white areas beyond the contour in Fig. 3.

We use Supervised AutoEncoder (SAE) [12] to generate 2D embeddings of references for projection. The decision to choose SAE over other dimensionality reduction techniques is based on two main reasons:

- First, SAE incorporates two losses - a reconstruction loss and a classification loss - which enable references to be labeled. This results in references with similar features and identical labels being positioned closely in the projection [55]. Consequently, we assign a label to each reference and newly collected GS for the purpose of projection. These labels are vital for enhancing the visualization and improving users' perception of visual patterns (see Section VI-D).
- Second, SAE supports pre-training. This allows us to train an SAE using references and then apply it to newly collected GSs that are co-distributed with the references without the need for retraining. This feature is essential for facilitating view operations. For instance, when users zoom into an area with limited references, we can gather new GSs based on existing references and progressively project them onto the plane (see Section VI-E).

We used all references as the training set for each dataset. SAE employs a three-layer encoder-decoder architecture with a two-dimensional bottleneck layer. The training process was efficient [55], typically converging within 100 epochs and completing in a few seconds.

Each point on the plane should correspond to a high-quality GS. We consider the task of identifying GSs associated with a particular position on the map as an optimization problem. Let c be the clicked position, z^* denote the latent vector of the GS to be optimized, and c^* indicate the location of z^* on the map produced by the SAE. We formulate the objective as:

$$f(z^*) = \arg\min_{z^*} \sqrt{\det G(z^*)} + \|c^* - c\|_2^2 , \tag{4}$$

where the first item ensures z^* to be on the manifold (the smaller $\det G(z^*)$, and the higher the probability that z^* is on the manifold), and the second item makes c^* as close to c as possible. We can utilize any optimization algorithm, such as gradient descent, to search for z^* from a GLS. With an infinite number of possible locations on the map, the Latent Space Map conforms to the Unboundedness (**C1**) and Generativity (**C2**) characteristics.

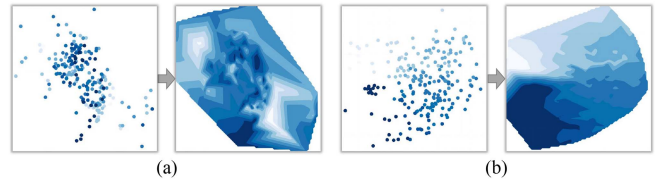

Fig. 4. Comparison of the projection with and without the classification loss. The color of each sample indicates its metric value. (a) Autoencoder (AE) without classification loss: The contours appear disorganized in terms of metric values. (b) Supervised AutoEncoder (SAE) with classification loss: Samples belonging to the same class are clustered in the same contour.

C. Metric Value Representation

We render a contour plot on the plane to display the distribution of the GSs on an application-oriented metric. The color at each position on the plane represents the application outcome of the corresponding GS, with darker colors indicating higher metric values, as shown in Fig. 3 (bottom). We choose the technique for two reasons. First, it is a classic visualization technique used on real-world maps to represent a continuous "field". It thus reduces users' learning burdens and improves pattern perception. Second, it improves rendering efficiency. We render it by interpolating across all references rather than calculating massive metric values for all 2D positions.

We employ Inverse Distance Weighting for interpolation [56] and Marching Squares for rendering the contour plot [57]. These algorithms offer two key advantages. First, Inverse Distance Weighting avoids assigning metric values to positions far from references with known metric values. Second, Marching Squares ensures that the contour plot only encompasses positions with metric values. Together, these algorithms provide a rough outline of the manifold on the plane. Regions without references appear white and can be viewed as dead zones, as shown in Fig. 3(a) (bottom). Users can focus on colored areas where GSs are likely to be from the manifold and of high quality. There are no restrictions on using these two algorithms, and users are free to substitute them with others having similar characteristics.

D. Perception Enhancement

To enhance users' perception, we optimize the visual representation of the contour plot by leveraging the classification loss of the SAE. While a traditional Autoencoder (AE) solely relies on reconstruction loss to organize GSs with their feature similarity, SAE introduces an additional classification loss. This guides GSs to be positioned on the plane according to both feature similarity and class labels, as demonstrated in Fig. 4. The core idea is to assign labels to each reference and project all references onto the plane using the SAE, utilizing their features and labels as inputs. This approach results in GSs with similar

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

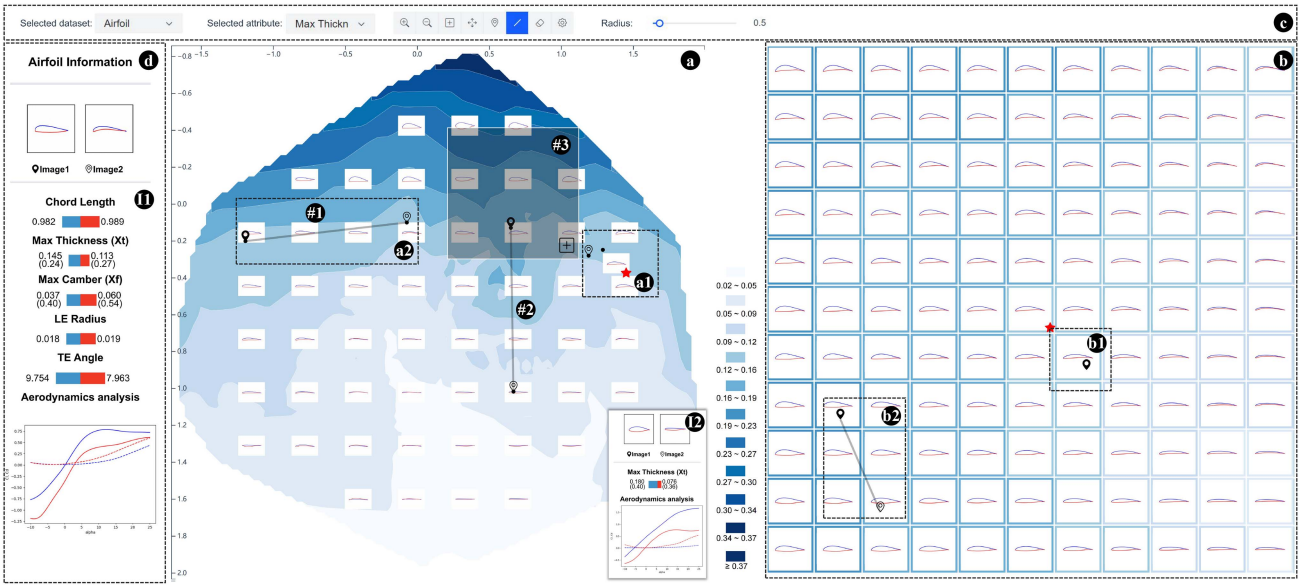

Fig. 5. The system interface primarily consists of two components: (a) Global View and (b) Local View. (c) Control Panel and (d) Information Panel are provided to facilitate user interactions and present supplementary information. Global View displays the application results of numerous GSs through a contour plot, where each position corresponds to a GS and the color represents the GS' value on a metric relevant to the application. Local View showcases GSs gathered around a target GS ★, with its two axes representing the first two dominant singular vectors that capture feature variations of the target.

features and matching labels being positioned close to each other on the plane (**C3**). Noting that labels are not obligatory, the arrangement of GSs is solely based on feature similarity, even without labels.

Specifically, we 1) discretize the metric values of references into intervals, with each interval representing a distinct class, and 2) assign a label to each reference based on the interval to which its metric value belongs. As the contour plot is generated using these metric values, GSs of the same color (indicating similar metric values) should form clusters, as demonstrated in Fig. 4(b). Here, we establish classes by segmenting the metric values of samples into 11 intervals, each representing a distinct class. The distribution displays a more structured color pattern, aiding in identifying the shared characteristics of samples with specific metric values. Thus, users can efficiently navigate regions with particular colors to identify GSs with either high or low metric values and discern their shared characteristics.

E. Consistent High-Accuracy Exploration

As a map, Latent Space Map should enable fundamental operations such as panning and zooming, enabling users to explore the manifold across various regions and scales. However, when users zoom into a small area with limited references, the accuracy of the contour plot in that area may decrease, as the plot is generated by interpolating metric values of the displayed references. There is a need to maintain high accuracy in the contour plot across all areas, regardless of the density of references present.

Fig. 3(b)–(c) illustrates the approach for achieving consistent high-accuracy exploration of Latent Space Map. When the user pans to a new area, a new set of GSs is gathered based on the references within the current display area, following Algorithm 1. Subsequently, the newly collected GSs can be incrementally projected onto the plane using the pre-trained SAE. Since the newly collected GSs are distributed alongside the references, there is no necessity to retrain the SAE. A new contour plot is then generated based on the newly gathered GSs to uphold high accuracy (**C5**).

VII. Visualization System

We design a visualization system incorporating Latent Space Map to promote its application in practice (**R3**). This section presents an overview of the system (Section VII-A), followed by detailed descriptions of the Global View (Section VII-B) and the Local View (Section VII-C).

A. System Overview

The visualization system is depicted in Fig. 5. The system primarily contains two components: a Global View and a Local View, which work together to support **R3**:

- Global View (Fig. 5(a)) is designed for users to interact with Latent Space Map. It enables users to explore any area of Latent Space Map and visualize application-oriented results of the collected GSs. This view ensures that all collected GSs are of high quality and supports operations to display the map at any region and scale.
- Local View (Fig. 5(b)) displays the GSs gathered around a selected target GS in the Global View, each displaying subtle feature variations from the target. Users can choose a GS with desirable feature variations to improve the target. Moreover, users have the option to adjust the region range to collect GSs with varying degrees of feature variations from the target GS.

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

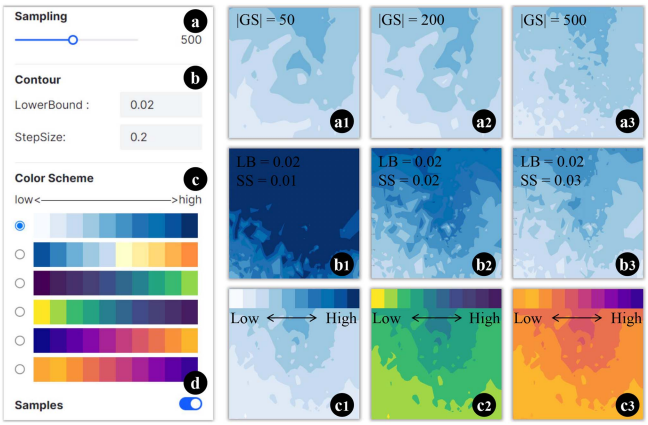

Fig. 6. Visual effect adjustments on four aspects: (a) the number of GSs to collect, (b) value intervals of contours, (c) color scheme, and (d) whether to overlap GSs on the Global View. The other nine subfigures show examples of the first three aspects under different settings.

The two views work in a coordinated manner. Users can either 1) choose a GS from the Global View to view its neighbors in the Local View, or 2) select a GS from Local View to observe its position in the Global View, and discover a new area with GSs exhibiting more desirable features to initiate another round of exploration. Through this approach, users can progressively identify regions abundant in desirable GSs by leveraging both views. Along with the two views, a Control Panel (Fig. 5(c)) facilitates user interactions within two views, and an Information Panel (Fig. 5(d)) presents supplementary information about GSs, tailored to different application scenarios.

B. Global View

Global View provides a visualization of Latent Space Map, as shown in Fig. 5(a). Each position on the plot represents a GS sampled from the manifold, with the color encoding the value of the GS on an application-oriented metric.

Interaction: Global View offers various user interactions, by selecting a button in the Control Panel as depicted in Fig. 5(c). There are four operations available to enable the adjustments of manifold range and redraw contour plot: zoom in, zoom out, box zoom, and pan . Besides the map operations, users can select GS for in-depth examination and comparison, by clicking 1) to select a single GS (Fig. 5(a1)), 2) to select two GSs for comparison (Fig. 5(a2)), and 3) to clear symbols of selected GSs. Upon selection, detailed information of application-oriented metrics of the selected GSs will be displayed in the Information Panel, as depicted in Fig. 5(d).

Visual Effect Adjustment: Various parameters, such as sampling rate and value intervals in the contour plot, can impact the visual representation of Latent Space Map and impact user exploration. The system offers operations for adjusting visual effects. Users can click on the icon to open a popup window where users can adjust the visual effects of Global View, as shown in Fig. 6. The system currently allows for the adjustment of four visual effects:

- Users can adjust the number of GSs collected for redrawing the contour plot, by using a slider as depicted in Fig. 6(a). The GSs are collected based on the references within the current display range. Global View will then redraw the contour plot with the references and the newly collected GSs. Increasing the number of GSs refines the contours, providing a more accurate representation of the manifold boundaries on the plane, but it also increases the computational burden on the back-end. Fig. 6(a1)–(a3) shows examples of contour plots with different numbers of GSs.
- Second, users can set LowerBound and StepSize parameters to regulate the value interval of each contour, as shown in Fig. 6(b). We utilize eleven fixed-width contours to simplify development. LowerBound determines the starting point of the value range represented by the contours, while StepSize influences the granularity of the patterns. Users can adjust StepSize to observe broader macro patterns across a wider range or finer-grained patterns within a narrower range. Fig. 6(b1)–(b3) show examples in different parameter values.
- Third, users can choose a color scheme to assign a distinctive color to each contour, as demonstrated in Fig. 6(c). Fig. 6(c1)–(c3) shows examples of contour plots with different color schemes.
- Last, users can superimpose a group of GSs onto their positions in Global View, as shown in Fig. 5(a). Points are sampled at uniform intervals on the contour plot, and their corresponding GSs are computed using Equation 4. GSs in dead zones (white areas lacking references) are concealed. Users can toggle the visibility of the GSs using a switch in the popup window, as shown in Fig. 6(d).

C. Local View

Local View presents a group of GSs collected around a selected target GS marked with from Global View. These GSs are arranged in a grid layout, as shown in Fig. 5(b). The horizontal and vertical axes represent the two primary dimensions affecting feature variations of the target GS. Consequently, GSs in the Local View share similarities with the target but exhibit variations in certain features. Each GS is outlined with a color corresponding to its metric value, aiding in the easy identification and utilization of GSs.

Axes Definition: The arrangement of GSs in the Local View shall conform to the following requirements. First, GSs that are horizontally or vertically adjacent should exhibit similar features, aiding in easy comparison to identify optimal choices. Second, GSs should encompass a range of representative feature variations relative to the target, allowing users to conduct thorough comparisons to select the most suitable GS. To achieve these objectives, we 1) calculate two components that dominate the feature variances of the target GS as the two axes of Local View, and 2) generate a GS for each grid based on the center coordinates of the grid. Let $z \in \mathbb{R}^n$ be the latent vector of the selected target GS. By performing Singular Value Decomposition (SVD) on the Jacobian matrix $J_z \in \mathbb{R}^{m \times n}$ of z, we extract the first two singular vectors from the right singular matrix V. These vectors

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

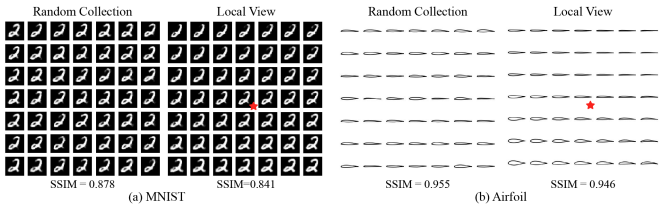

Fig. 7. Comparison of GSs randomly collected from the GLS and those generated within Local View on two datasets. In each dataset, both groups are gathered around the same target GS (marked with ★) with the same radius.

correspond to the two largest singular values, which indicate the directions along which GS features vary most significantly [58], [59] (see Appendix IV). Hence, we utilize these vectors as the two axes of Local View.

Sample Generation: Let v_x and v_y denote the two selected singular vectors, which are normalized to unit length. For each grid of Local View, we can derive a GS z' based on z using the following method:

$$z' = z + r * (xv_x + yv_y), \tag{5}$$

where x and y are the coordinates of a grid center in the Local View ($x, y \in [-1, 1]$), and r is the coordinate range ($r \in [0, +\infty)$). Users can adjust r to collect GSs with different degrees of feature variations from z.

Fig. 7 presents a comparison of GSs randomly collected from the GLS and those generated in the Local View on two datasets. GSs in the Local View exhibit consistent variations in features along the horizontal and vertical axes. Moreover, GSs generated in the Local View display a broader range of feature variations, evident from visual observations and their lower Structural Similarity Index Measure (SSIM) [60] compared to the target marked with .

Interaction: Local View offers the same GS selection interactions as Global View. Users can click on 1) to select a single GS, and 2) to select two GSs for comparison. By clicking on a GS in the Local View (Fig. 5(b1)), users can observe its corresponding position in the Global View (Fig. 5(a1)), and the detailed information of a selected GS is presented in the Information Panel.

D. Implementation

We build the system based on the VUE framework, with the front-end interface developed in D3.js. Generative models are trained using Pytorch. Flask is used to facilitate communication between the interface and the back-end. All algorithms, including reference-based sampling and singular vectors calculation, are executed server-side and implemented in Python, to manage users' sample requests and provide GSs along with their metrics within the specified area to the interface for visualization.

VIII. Evaluation

This section presents qualitative case studies (Section VIII-A) and quantitative experiments (Section VIII-B) to illustrate the system's effectiveness. Lastly, we provide feedback on the system usability obtained from expert interviews (Section VIII-C).

A. Case Study

We conducted two case studies on airfoil design (Section VIII-A1) and machine learning (Section VIII-A2), with experts E1 & E2 who participated in the preliminary study for requirement analysis. Each case study involved a face-to-face meeting with an expert in a designated meeting room. During each session, we first introduced the expert to our basic concept and system functionality. The expert then utilized the system freely to identify patterns, with an experimenter available to address any questions from the expert.

1) Study 1. Airfoil Design: We trained a VAE on the UIUC Airfoil Coordinates Database [61], which contains outline coordinates for over 1600 airfoils. Geometric attributes (Fig. 9(a)) are used to assign a category for each airfoil (Fig. 9(b)–(e)). Users can select any geometric attribute to create classes by dividing the value range into intervals. Airfoils with similar geometric attributes are placed adjacent to each other on Latent Space Map, enabling users to identify airfoils with specific attribute values and optimal aerodynamic performance (Fig. 9(f)).

Global Pattern: E1 aimed to design airfoils with high lift and low drag coefficients. Considering that the maximum thickness influences these coefficients, E1 categorized each airfoil based on its maximum thickness. Latent Space Map in Fig. 5(a) illustrates the classification results, with bluer shades indicating higher maximum thickness. E1 observed that: 1) airfoils in the lower section of the plot tended to be reflex-cambered (with smaller thickness), 2) airfoils in the upper regions were predominantly cambered or biconvex (with greater thickness), and 3) symmetric airfoils with median thickness were concentrated in the center.

These findings motivated E1 to investigate the lift coefficient across areas with different colors on Latent Space Map. E1 proceeded to compare airfoils. He first drew a horizontal line to compare airfoils with the approximate maximum thickness (Fig. 5(#1)). It was observed that the biconvex airfoil (indicated by) exhibited higher lift coefficients compared to the cambered airfoil (indicated by), with their information depicted by the red and blue colors in Fig. 5(I1). Next, E1 drew a vertical line to compare airfoils with different maximum thicknesses (Fig. 5(#2)) and found airfoils with greater thickness displayed increased lift coefficients (Fig. 5(I2)).

Local Refinement: These findings motivated E1 to focus on a specific region with biconvex airfoils of greater thickness in Fig. 5(#3) and zoom in on this region that derives Fig. 8(a). The color represented airfoil thicknesses ranging from 0.12 to 0.27. Recognizing that the color interval was broad, E1 adjusted the StepSize to 0.01 and LowerBound to 0.14, to differentiate thicknesses within the range, as shown in Fig. 8(b). After analyzing the airfoil information, E1 selected an airfoil with median thickness (marked with) exhibiting a higher lift coefficient and utilized Local View to refine the design.

Local View (Fig. 8(c)) shows airfoils collected around the target airfoil. Through pairwise comparisons, E1 observed that

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

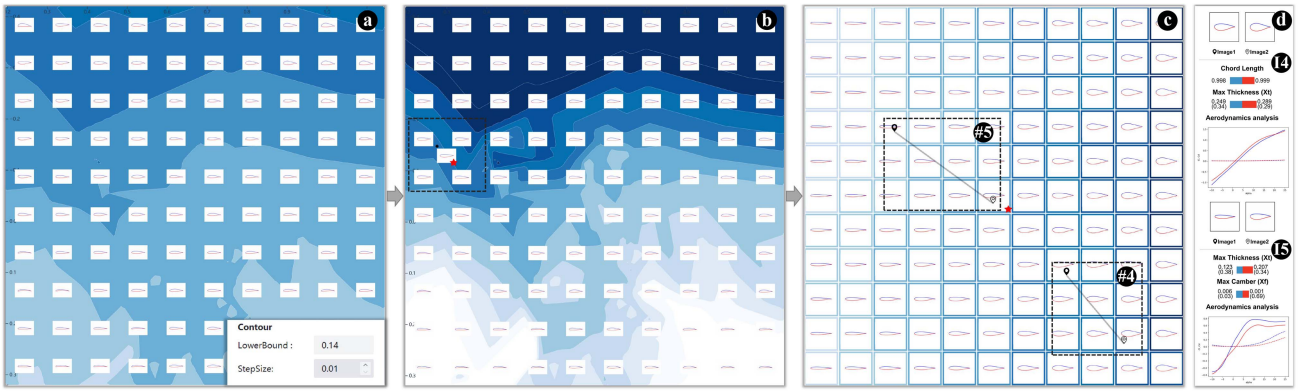

Fig. 8. Case study on airfoil design. (a) Global View after zooming into #3 of Figure 5(a). (b) Global View after setting a large LowerBound and small StepSize of (a). (c) Local View showed images collected around a target GS selected in (b) (marked with ★). (d) Information of images selected in (c).

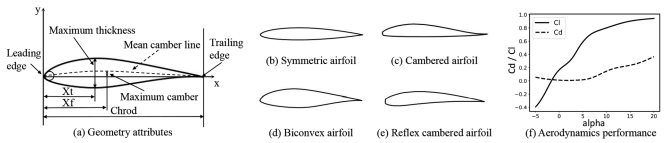

Fig. 9. Background knowledge of airfoils, including (a) geometric attributes and (b-e) the main airfoil types. (f) Aerodynamics performance of an airfoil calculated by NeuralFoil tool [62]. The solid and dashed lines depict the lift (Cl) and drag (Cd) coefficients at various attack angles.

1) the benefits of increasing the lift coefficient diminish as the maximum thickness continues to increase beyond a certain point, as in Fig. 8(#4) and (I4); and 2) enhancing the maximum camber simultaneously improved the lift and drag coefficients, even with lower maximum thickness, as in Fig. 8(#5) and (I5). E1 selected a few biconvex airfoils with greater camber, and designated the maximum camber as the metric for the subsequent round of investigation.

2) Study 2. Classifier Diagnosis and Improvement: E2 employed our system to evaluate an image classifier (ResNet18) trained on the MNIST dataset. We trained a GAN model using MNIST, with the original ten classes (numbers 0-9) serving as labels. GSs representing the same number were positioned adjacent to each other on the Latent Space Map. We chose a red-blue color scheme, where correct classifications are depicted in red and incorrect classifications are in blue. The darker the color at a position, the higher the classifier confidence of the associated GS. The presence of a dark blue region signifies consistent errors by the classifier when processing the involved GSs, highlighting potential deficiencies.

Deficiency Diagnostic: E2 found that most regions in the Global View were colored red, indicating that the classifier achieved an overall high accuracy (96.7%), as depicted in Fig. 10(a). Nonetheless, a few small regions appeared blue. Upon closer examination of these regions, E2 identified that the classifier exhibited lower accuracy at the boundaries between specific classes, such as ‘4’ and ‘9’ (#1), ‘7’ and ‘9’ (#2), and ‘3’ and ‘5’ (#3). E2 zoomed into region #3 and noted that numerous GSs in this area shared similarities with ‘1’, ‘3’, and

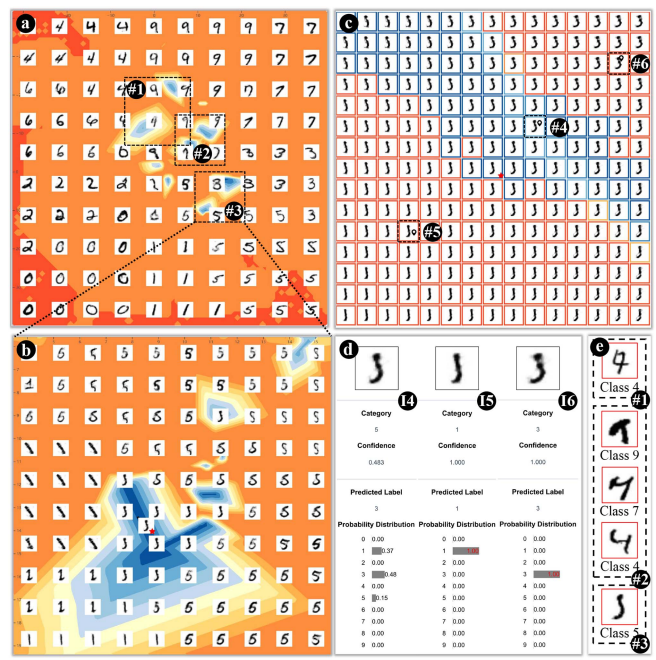

Fig. 10. Case study on classifier diagnosis and enhancement. (a) The initial Global View indicates three regions (#1-#3) containing misclassified images. (b) Global View after zooming in on region #3. (c) Local View displaying images gathered around a selected target GS marked with ★ in (b). (d) Details of the images chosen in (c). (e) Training examples resembling the images from the three regions in (a).

‘5’ simultaneously, resulting in the poor performance of the classifier, as depicted in Fig. 10(b).

Feature Effects: E2 selected a misclassified GS (marked with in the Global View (Fig. 10(b)) and examined its neighboring GSs in the Local View to investigate the contributing factors to their misclassification, as shown in Fig. 10(c). Local View displayed three distinct regions: the lower-left region containing images of ‘1’ (Fig. 10(#5)), the upper-right region consisting of images of ‘3’ (Fig. 10(#6)), and the central region comprising images of ‘5’ (Fig. 10(#4)). Notably, GSs in regions #5 and #6 were accurately classified, whereas those in region #4 were consistently misclassified. E2 observed that the hooks at the

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

lower part of images in region #4 (I4) exhibited less curvature compared to images in region #6 (I6), leading to a potential impact on number identification. This observation is further elaborated in Fig. 10(d).

Defect Analysis and Accuracy Improvement: Fig. 10(e) shows training samples that are similar to those misclassified within the three regions. These samples serve as evidence that our approach can gather high-quality GSs that closely mirror the misclassified images, thereby elucidating the classifier's poor performance on images from these regions. For instance, we identify training samples resembling the numbers '7', '9', and '4' (regions #1 and #2 in Fig. 10(e)), which lead to the classifier's confusion between these three digits. Furthermore, we observe notably irregular samples of the digit '5' that exhibit similarities to digits '1' and '3' to some degree (region #3 in Fig. 10(e)), resulting in the misclassification of similar GSs. E2 generated GSs for each class, selected those with medium confidence scores, and curated 50 high-quality GSs per class. GSs were incorporated into the training set (generated-to-real ratio $= 1:120$), enriching the decision boundary regions and increasing classifier accuracy from 96.7% to 97.3% after retraining.

B. Quantitative Experiment

Reference-based sampling is a fundamental component of Latent Space Map. We conducted quantitative experiments to demonstrate the usability and effectiveness of our reference-based sampling method. Specifically, the method was compared with the commonly used Gaussian prior-based method [1], [2] and a recent Monte Carlo-based (MC-based) method [28]. The MC-based method is similar to ours but specifically tailored for VAEs. All experiments were conducted on a server equipped with an Intel Xeon Gold CPU with 5218R, 512 GB of memory, and an NVIDIA RTX 3090 GPU.

GS Quality: We compared the quality of GSs collected using the three methods on six datasets: UIUC Airfoil Coordinates [61], MNIST [63], Fashion [64], Cifar10 [65], Cifar100 [65] and Tiny ImageNet [66]. The first two datasets have been utilized in case studies, while the latter are recognized as classic deep-learning benchmarks. We trained VAEs on Airfoil and Fashion, SNGAN [67] on MNIST, BigGAN [68] on Cifar10, StyleGAN2 [69] on Cifar100, and ECGAN [70] on Tiny ImageNet. GSs were collected from the GLS of each model using the different methods, with the sample size of 10 k. We calculate the Fréchet Inception Distance (FID) [71] and Inception Score (IS) [72] for each group of GSs, where a lower FID and a higher IS indicate better quality and greater feature diversity of the GSs.

Table I presents the experiment results. Our reference-based method consistently exhibits lower FID and higher IS compared to the Gaussian prior-based method across all conditions, indicating its usability and effectiveness. Ours and the MC-based method demonstrate comparable performance. However, the model-agnostic nature of our method enhances its applicability. We conducted a qualitative experiment on the distribution of GSs, presenting that our method collects GSs with a distribution similar to the references (see Appendix VI).

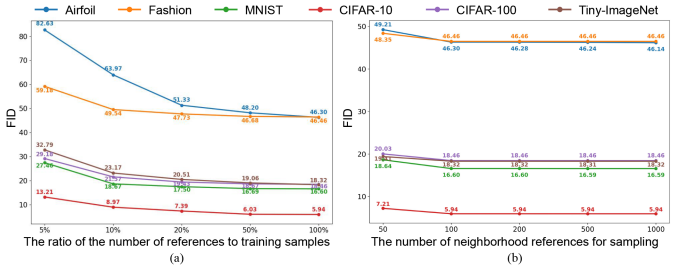

Fig. 11. Sampling effects under the influence of two parameters, i.e., (a) the number of references (the ratio of it to that of the training set) and (b) the number of neighborhood references for computing $G(z)$.

Effects of the Number of References: We evaluated the effects of the number of references used in constructing the manifold for sampling. We randomly collect references from the training set and ensure they uniformly cover each class. To facilitate comparisons, we use the ratio of the number of references to that of training samples as the variable and evaluate how the FID of the collected GSs (10 K) varies with the variable, ranging from 5% to 100%.

Fig. 11 (a) displays the experiment results. Notably, when the number of references encompasses all training samples, FID is minimized. Besides, the rate of decrease in FID levels off. This indicates that our method is particularly effective for smaller datasets, where all training samples can serve as references. For large datasets, such as those with millions of samples, it is also feasible to sample a representative subset to construct the manifold.

Effects of the Number of Neighborhood References: The computation of Riemannian metric $G(z)$ depends on the number of neighborhood references N (see Equation 1). We evaluated the impact of N on the sampling method, ranging from 50 to 1000. We collected 10k GSs and calculated their FID on different datasets.

Fig. 11(b) presents the results. When $N \geq 100$, increasing N has minimal impact on FID. This aligns with the definition of $G(z)$, where distant references are assigned less weight. Therefore, we set $N = 100$ for all experiments, which reduces computational overhead while maintaining sample diversity.

Effects of GLS Dimensionality and Jacobian Matrix Pre-calculation: We evaluated the computation times of our reference-based method with and without pre-calculating the Jacobian matrices of references. We trained VAEs with GLSs of varying dimensions (d ranging from 10 to 512) on the Airfoil dataset, and collected varying numbers of GSs S from the GLS of each model (S ranging from 50 to 500). When $d = 512$, the memory of one GPU was exceeded, so two GPUs were used. The sampling times across different combinations of S and d were calculated.

Table II presents the experiment results. The computation time increases as d increases, and it remains acceptable even when $d = 512$. d does not significantly affect our approach in most cases, as it is typically not very large (see Table I). Pre-calculation results in significantly reduced sampling time. The computation of $G(z)$ involves time-consuming layer-by-layer derivatives of the Jacobian matrices [54], leading to lower

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

efficiency. We measured the average computation time required to compute a single Jacobian matrix. For MNIST (28×28, $d = 32$) and Tiny-ImageNet ($32 \times 32 \times 3$, $d = 100$), the computation requires approximately 0.27 and 6.74 s, respectively. Our current GPU (NVIDIA RTX 3090) can pre-process tens of thousands of samples within a few hours, which is sufficient for most scenarios. For larger datasets, more powerful GPU resources can be employed to complete the pre-computation within an acceptable timeframe. This highlights the importance and effectiveness of our optimizations (Section V-A) in enhancing sampling efficiency.

C. Expert Interview

We conducted interviews with three experts to gather their feedback on the system's usability. Along with E1 & E2 who participated in the case study, we also engaged a project manager from a leading AI company (E3, male, aged 38) who was overseeing an autonomous driving project focused on training decision-making models with generated data.

Study Procedure: For E1 and E2, we gathered their feedback in-person immediately after each case study. An experimenter documented their feedback. The interview with E3 was conducted remotely using an online conferencing software. To aid E3 in quickly grasping the idea and system usage, we prepared a video showcasing the two cases. E3 had the ability to pause the video to ask questions or offer suggestions on specific details. An experimenter was tasked with addressing his inquiries and recording his feedback. The interview session lasted approximately 20 minutes.

Feedback: All experts agreed that the visual design is intuitive and user-friendly. E1 remarked, "*The visual design is straightforward and facilitates easy navigation of GSs.*" E2 appreciated that "*the interactions are logical, creating a browsing experience like navigating a map to discover GSs within a GLS.*" E3 particularly appreciated the cohesive design of Global View and Local View, as "*I can readily understand how to utilize the two views to identify GSs of interest.*"

The experts confirmed the usability of the system in various scenarios. In the context of airfoil design, the system enables users to filter airfoils based on specific geometric attributes in the Global View and make adjustments in the Local View. E1 appreciated the effectiveness of Local View. "*We often encounter design adjustments where we lack clarity, and Local View presents numerous feasible solutions with improved aerodynamic performance. This allows us to easily refine the design.*" In terms of model evaluation, E2 believed that the system is well-suited for testing machine learning models. He elaborated, "*Global View enables easy identification of misclassified GSs, while Local View displays GSs that deviate from a chosen misclassified GS. These functions are crucial for model testing and challenging to implement without a visualization system.*" E2 further noted, "*The system is versatile. Users can change the metric for visualizing the model's performance in other aspects, such as robustness.*"

E3 recognized the concept behind creating a Latent Space Map to utilize GSs. He expressed, "*It is impractical to gather samples that vary slightly from real-world data to identify reasons for prediction errors. Utilizing generated data is a feasible way.*" E3 also noted, "*The system has the capability to filter out low-quality GSs, a prerequisite for effective utilization of GSs.*" In addition, E3 emphasized the system's high responsiveness as essential for practical application. He agreed with the way of pre-computing Jacobian matrices to improve the system's response speed.

However, experts highlighted some limitations of the system. E1 suggested incorporating professional functions in specific scenarios. He pointed out, "*The system lacks professional functions such as adjusting geometric parameters and evaluating detailed aerodynamic performance, which is vital for airfoil design.*" E1 further noted, "*The cases validate the usability of utilizing infinite GSs, indicating success in meeting the framework's design goals. However, for scenarios that require intricate domain knowledge, the integration of more professional functions is crucial.*" Nevertheless, he noted that this is not specific to our system but a common challenge in connecting deep learning with practical applications.

The experts also provided many suggestions. E1 proposed the search function within the system "*can easily locate GSs with specific attribute values*". E2 recommended enhancing Latent Space Map to encompass model comparison scenarios, allowing for direct color comparisons within the same regions. E3 emphasized incorporating additional engineering functions, such as data import and result export features, to improve the overall usability and functionality of the system.

IX. Discussion

The evaluation has shown the efficacy of our approach in both the Latent Space Map and system design. However, there are limitations that present opportunities for future research.

Adaptation to Other Generative Models: Diffusion models (DMs) have the capability to generate high-quality images. However, our approach has not been applied to DMs due to certain limitations, including 1) DMs are generally slower than VAEs and GANs, making them unsuitable for real-time systems, and 2) calculating the Jacobian matrix in DMs requires a significant amount of memory due to the large input and output sizes. The development of new techniques, such as Latent Diffusion Model [73], makes it possible to overcome these challenges. We plan to explore the feasibility of applying our approach to DMs using these innovative techniques.

Large language models (LLMs) fall under the category of generative models. Open-source LLMs (e.g., LLaMA [74]) provide transparency into their architectures, enabling us to select an intermediate layer as GLS for implementing our approach. In contrast, closed-source LLMs conceal their internal workings. To apply our approach to closed-source LLMs, one potential solution is to train a separate encoder based on these models [75], and then apply our approach to the latent space generated by the encoder. Further endeavors are required to formulate a solution in this realm.

Data Distribution Enrichment: Generative models are designed to learn the distribution of training samples, thereby

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

that GSs align with this distribution. However, this can pose a challenge when using GSs for model testing, as any flaws in the model may not be evident in a deployment environment where the sample distribution differs from the training sample. To address this issue, we are exploring the concept of introducing perturbations into the generation process to diversify the distribution of the synthesized data from that of the training samples. It is crucial to carefully control these perturbations to ensure that they do not compromise the quality of the generated samples. Inspired by open-set studies, such as those outlined in [76], we plan to leverage similar techniques to extend the functionality of Latent Space Map.

Manifold Optimization: GLS is unbounded and encompasses infinite samples. It is impossible to identify all high-quality regions and map them onto a plane. Our approach constructs the manifold based on the references derived from the training set, aiming to capture the representative features of these references. Thus, the quality of the collected samples is close to that of the references. While increasing the number of references improves the quality of the samples, this gain will continuously decrease. Moreover, it is impractical to use too many references, as this would overburden the initialization phase with calculating their Jacobian matrices. Future work may explore a new theoretical framework that establishes a bidirectional mapping between GLS and Latent Space Map, similar to the approach used in iLAMP for LAMP [37]. This could incorporate a metric-based filtering strategy to identify all high-quality GSs within the GLS [77].

X. Conclusion

This study introduces a novel visualization approach designed to facilitate the utilization of GS. The idea is to construct a Latent Space Map of a GLS to assist users in locating areas with desirable features and application-oriented metrics. The design of Latent Space Map follows specific criteria, including 1) to share common characteristics with the GLS, 2) to project the manifold of high-quality GSs onto a plane rather than the whole GLS for validity, and 3) to enable multi-scale exploration of any map area while maintaining a high level of display accuracy. We develop an interactive visualization system based on Latent Space Map for practical applications in airfoil design and model development. Findings from real-world case studies, quantitative experiments, and expert feedback confirm the usability and effectiveness of our approach, highlighting the potential for further research utilizing GSs with interactive visualization.

References

[1] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. Int. Conf. Learn. Representations*, 2014.

[2] I. Goodfellow et al., "Generative adversarial networks," *Commun. ACM*, vol. 63, no. 11, pp. 139–144, 2020.

[3] L. Zheng, K. Karapiperis, S. Kumar, and D. M. Kochmann, "Unifying the design space and optimizing linear and nonlinear truss metamaterials by generative modeling," *Nature Commun.*, vol. 14, no. 1, 2023, Art. no. 7563.

[4] A. Jahanian, X. Puig, Y. Tian, and P. Isola, "Generative models as a data source for multiview representation learning," in *Proc. Int. Conf. Learn. Representations*, 2022.

[5] Y. Zhang, J. Li, and C. Xu, "Graph-based latent space traversal for new molecules discovery," in *Proc. Int. Symp. Vis. Inf. Commun. Interaction*, 2023, pp. 1–8.

[6] B. C. Kwon et al., "Latent space explorer: Visual analytics for multimodal latent space exploration," 2023, *arXiv:2312.00857*.

[7] Z. Xiao, K. Kreis, and A. Vahdat, "Tackling the generative learning trilemma with denoising diffusion GANs," in *Proc. Int. Conf. Learn. Representations*, 2022.

[8] T. White, "Sampling generative networks," 2016, *arXiv:1609.04468*.

[9] M. Girolami and B. Calderhead, "Riemann manifold Langevin and Hamiltonian Monte Carlo methods," *J. Roy. Stat. Soc. Ser. B, Stat. Methodol.*, vol. 73, no. 2, pp. 123–214, 2011.

[10] Y. Fei, Y. Liu, C. Jia, Z. Li, X. Wei, and M. Chen, "A survey of geometric optimization for deep learning: From Euclidean space to Riemannian manifold," *ACM Comput. Surv.*, vol. 57, no. 5, pp. 1–37, 2025.

[11] H. Shao, A. Kumar, and P. Thomas Fletcher, "The Riemannian geometry of deep generative models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2018, pp. 315–323.

[12] L. Le, A. Patterson, and M. White, "Supervised autoencoders: Improving generalization performance with unsupervised regularizers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 107–117.

[13] Y. Liu, E. Jun, Q. Li, and J. Heer, "Latent space cartography: Visual analysis of vector space embeddings," *Comput. Graph. Forum*, vol. 38, no. 3, pp. 67–78, 2019.

[14] O.-H. Kwon and K.-L. Ma, "A deep generative model for graph layout," *IEEE Trans. Vis. Comput. Graph.*, vol. 26, no. 1, pp. 665–675, Jan. 2020.

[15] M. Kahng, N. Thorat, D. H. Chau, F. B. Viégas, and M. Wattenberg, "GAN lab: Understanding complex deep generative models using interactive visual experimentation," *IEEE Trans. Vis. Comput. Graph.*, vol. 25, no. 1, pp. 310–320, Jan. 2019.

[16] G. Loaiza-Ganem, B. L. Ross, R. Hosseinzadeh, A. L. Caterini, and J. C. Cresswell, "Deep generative models through the lens of the manifold hypothesis: A survey and new connections," *Trans. Mach. Learn. Res.*, 2024.

[17] A. N. Gorban and I. Y. Tyukin, "Blessing of dimensionality: Mathematical foundations of the statistical physics of data," *Philos. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 376, no. 2118, 2018, Art. no. 20170237.

[18] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4401–4410.

[19] J. Tomczak and M. Welling, "VAE with a VampPrior," in *Proc. 21st Int. Conf. Artif. Intell. Statist.*, 2018, pp. 1214–1223.

[20] E. Nalisnick, L. Hertel, and P. Smyth, "Approximate inference for deep latent Gaussian mixtures," in *Proc. NIPS Workshop Bayesian Deep Learn.*, vol. 2, 2016, p. 131.

[21] X. Chen et al., "Variational lossy autoencoder," in *Proc. Int. Conf. Learn. Representations*, 2017.

[22] B. Pang, T. Han, E. Nijkamp, S.-C. Zhu, and Y. N. Wu, "Learning latent space energy-based prior model," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 21994–22008.

[23] G. Arvanitidis, L. K. Hansen, and S. Hauberg, "Latent space oddity: On the curvature of deep generative models," in *Proc. Int. Conf. Learn. Representations*, 2018.

[24] A. Voynov and A. Babenko, "Unsupervised discovery of interpretable directions in the GAN latent space," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 9786–9796.

[25] Y. Shen and B. Zhou, "Closed-form factorization of latent semantics in GANs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1532–1540.

[26] A. Creswell, K. Arulkumaran, and A. A. Bharath, "Improving sampling from generative autoencoders with Markov chains," *Comput. Res. Repository*, 2016.

[27] R. M. Neal, "MCMC using Hamiltonian dynamics," in *Handbook of Markov Chain Monte Carlo*, 1st ed., S. Brooks, A. Gelman, G. Jones, and X.-L. Meng, Eds., London, U.K.: Chapman and Hall/CRC, 2011, pp. 113–162. [Online]. Available: https://www.taylorfrancis.com/chapters/edit/10.1201/b10905-6/mcmc-using-hamiltonian-dynamics-radford-neal

[28] C. Chadebec, E. Thibeau-Sutre, N. Burgos, and S. Allassonnière, "Data augmentation in high dimensional low sample size setting using a geometry-based variational autoencoder," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2879–2896, Mar. 2023.

[29] C. Ma and X. Zhang, "GF-VAE: A flow-based variational autoencoder for molecule generation," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 1181–1190.

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

[30] P. Z. Wang and W. Y. Wang, "Riemannian normalizing flow on variational wasserstein autoencoder for text modeling," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2019, pp. 284–294.

[31] Y. Ye, S. Xiao, X. Zeng, and W. Zeng, "ModalChorus: Visual probing and alignment of multi-modal embeddings via modal fusion map," *IEEE Trans. Vis. Comput. Graph.*, vol. 31, no. 1, pp. 294–304, Jan. 2025.

[32] L. G. Nonato and M. Aupetit, "Multidimensional projection for visual analytics: Linking techniques with distortions, tasks, and layout enrichment," *IEEE Trans. Vis. Comput. Graph.*, vol. 25, no. 8, pp. 2650–2673, Aug. 2019.

[33] J. Matejka, M. Glueck, E. Bradner, A. Hashemi, T. Grossman, and G. Fitzmaurice, "Dream lens: Exploration and visualization of large-scale generative design datasets," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2018, pp. 1–12.

[34] T. Cabezon Pedroso and D. Byrne, "Browsing the latent space: A new approach to interactive design exploration for volumetric generative systems," in *Proc. 15th Conf. Creativity Cogn.*, 2023, pp. 330–333.

[35] H. Dang and D. Buschek, "GestureMap: Supporting visual analytics and quantitative analysis of motion elicitation data by learning 2D embeddings," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2021, pp. 1–12.

[36] M. Padala, D. Das, and S. Gujar, "Effect of input noise dimension in GANs," in *Proc. Int. Conf. Neural Inf. Process.*, 2021, pp. 558–569.

[37] E. P. dos Santos Amorim, E. V. Brazil, J. Daniels, P. Joia, L. G. Nonato, and M. C. Sousa, "iLAMP: Exploring high-dimensional spacing through backward multidimensional projection," in *Proc. IEEE Conf. Vis. Analytics Sci. Technol.*, 2012, pp. 53–62.

[38] P. Joia, D. Coimbra, J. A. Cuminato, F. V. Paulovich, and L. G. Nonato, "Local affine multidimensional projection," *IEEE Trans. Vis. Comput. Graph.*, vol. 17, no. 12, pp. 2563–2571, Dec. 2011.

[39] E. Amorim et al., "Facing the high-dimensions: Inverse projection with radial basis functions," *Comput. Graph.*, vol. 48, pp. 35–47, 2015.

[40] L. Gou et al., "VATLD: A visual analytics system to assess, understand and improve traffic light detection," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 2, pp. 261–271, Feb. 2021.

[41] Q. Wang, S. L'Yi, and N. Gehlenborg, "DRAVA: Aligning human concepts with machine learning latent dimensions for the visual exploration of small multiples," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2023, pp. 1–15.

[42] H. Ohno, "Auto-encoder-based generative models for data augmentation on regression problems," *Soft Comput.*, vol. 24, no. 11, pp. 7999–8009, 2020.

[43] A. Kebaili, J. Lapuyade-Lahorgue, and S. Ruan, "Deep learning approaches for data augmentation in medical imaging: A review," *J. Imag.*, vol. 9, no. 4, 2023, Art. no. 81.

[44] Y. Ye et al., "Generative AI for visualization: State of the art and future directions," *Vis. Inform.*, vol. 8, no. 2, pp. 43–66, 2024.

[45] S. Shao, P. Wang, and R. Yan, "Generative adversarial networks for data augmentation in machine fault diagnosis," *Comput. Ind.*, vol. 106, pp. 85–93, 2019.

[46] C. Shivashankar and S. Miller, "Semantic data augmentation with generative models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 863–873.

[47] V. A. Fajardo et al., "On oversampling imbalanced data with deep conditional generative models," *Expert Syst. Appl.*, vol. 169, 2021, Art. no. 114463.

[48] A. Burnap, Y. Liu, Y. Pan, H. Lee, R. Gonzalez, and P. Y. Papalambros, "Estimating and exploring the product form design space using deep generative models," presented at ASME 2016 Int. Des. Eng. Tech. Conferences Comput. Inf. Eng. Conf., Charlotte, NC, USA, Aug. 21–24,, vol. 50107, 2016, Paper V02AT03A013.

[49] S. Khan and M. J. Awan, "A generative design technique for exploring shape variations," *Adv. Eng. Inform.*, vol. 38, pp. 712–724, 2018.

[50] L. Regenwetter, B. Curry, and F. Ahmed, "BIKED: A dataset for computational bicycle design with machine learning benchmarks," *J. Mech. Des.*, vol. 144, no. 3, 2022, Art. no. 031706.

[51] J. M. Lee, *Riemannian Manifolds: An Introduction to Curvature*, vol. 176. New York, NY, USA: Springer, 2006.

[52] M. Louis, "Computational and statistical methods for trajectory analysis in a Riemannian geometry setting," Ph.D. dissertation, Sorbonne Université, Paris, France, 2019.

[53] W. Xia, Y. Zhang, Y. Yang, J.-H. Xue, B. Zhou, and M.-H. Yang, "GAN inversion: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3121–3138, Mar. 2023.

[54] N. Chen, A. Klushyn, R. Kurle, X. Jiang, J. Bayer, and P. Smagt, "Metrics for deep generative models," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2018, pp. 1540–1550.

[55] J. Li and C.-q. Zhou, "Incorporation of human knowledge into data embeddings to improve pattern significance and interpretability," *IEEE Trans. Vis. Comput. Graph.*, vol. 29, no. 1, pp. 723–733, Jan. 2023.

[56] D. W. S. Wong, "Interpolation: Inverse-distance weighting," *Int. Encyclopedia Geogr.: People, Earth, Environ. Technol.*, Wiley, pp. 1–7, 2016, doi: 10.1002/9781118786352.wbieg0066.

[57] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," in *Proc. Conf. Comput. Graph. Interactive Techn.*1987, vol. 21, no. 4, pp. 163–169.

[58] Y. Wei et al., "Orthogonal Jacobian regularization for unsupervised disentanglement in image generation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 6721–6730.

[59] J. Choi, J. Lee, C. Yoon, J. H. Park, G. Hwang, and M. Kang, "Do not escape from the manifold: Discovering the local coordinates on the latent space of GANs," in *Proc. Int. Conf. Learn. Representations*, 2021.

[60] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[61] M. Selig, "UIUC airfoil data site," United States, 1996.

[62] P. Sharpe, "NeuralFoil: An airfoil aerodynamics analysis tool using physics-informed machine learning," 2023. [Online]. Available: https://github.com/peterdsharpe/NeuralFoil

[63] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[64] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.

[65] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," M.S. thesis, Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 2009.

[66] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. 2009 IEEE Conf. Comput. Vis. Pattern Recognit., 2009, pp. 248–255.

[67] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *Proc. Int. Conf. Learn. Representations*, 2018.

[68] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," in *Proc. Int. Conf. Learn. Representations*, 2019.

[69] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, "Analyzing and improving the image quality of styleGAN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8110–8119.

[70] S.-A. Chen, C.-L. Li, and H.-T. Lin, "A unified view of cgans with and without classifiers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 27566–27579.

[71] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 6626–6637.

[72] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 2234–2242.

[73] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10684–10695.

[74] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[75] S. Paun, "Variational auto-encoder for latent uncertainty encoding in large language models," M.S. thesis, Univ. Twente, Enschede, the Netherlands, 2025.

[76] M. Salehi, H. Mirzaei, D. Hendrycks, Y. Li, M. H. Rohban, and M. Sabokrou, "A unified survey on anomaly, novelty, open-set, and out-of-distribution detection: Solutions and future challenges," *Trans. Mach. Learn. Res.*, 2022.

[77] J. Xu, H. Le, and D. Samaras, "Assessing sample quality via the latent space of generative models," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 449–464.

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl

Yang Zhang is currently working toward the master's degree with the College of Intelligence and Computing, Tianjin University. She has authored or coauthored several papers in visualization and now works on visualizing the high-dimensional latent space of DNNs.

Jie Li is currently a full professor with the College of Intelligence and Computing, Tianjin University. His research interests include visual analytics, particularly on temporal, textual, and multidimensional data. With the support of NSFC projects, he is currently working on using visualization to improve the usability of AI systems. He has won several paper awards in visualization and authored or coauthored more than ten papers in *IEEE Transactions on Visualization and Computer Graphics*, IEEE VIS, and EuroVIS. His academic homepage is http://geoanalytics.tju.edu.cn/jieli/.

Wei Zeng received the PhD degree in computer science from Nanyang Technological University in 2015. He is currently an assistant professor with the Hong Kong University of Science and Technology (Guangzhou). His recent research interests include visualization and visual analytics, and AIGC. He was the recipient of the Best Paper Awards from ICIV, VINCI, and ChinaVis. He was the program chair for VINCI'23, and program committee for venues including IEEE VIS, EuroVis STARs, and ChinaVis.

uthorized licensed use limited to: The Hong Kong University of Science and Technology (Guangzhou). Downloaded on December 31,2025 at 15:09:56 UTC from IEEE Xplore. Restrictions appl